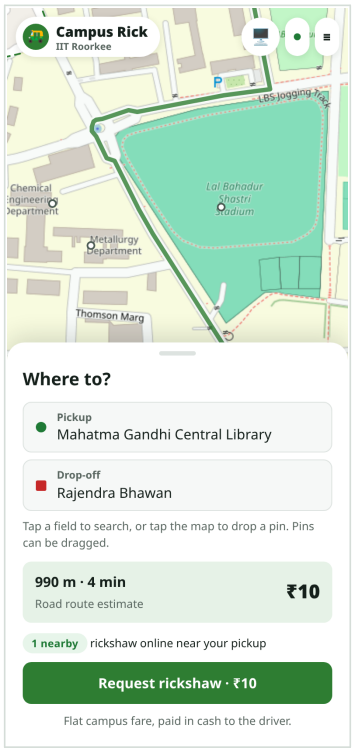
Campus Rick

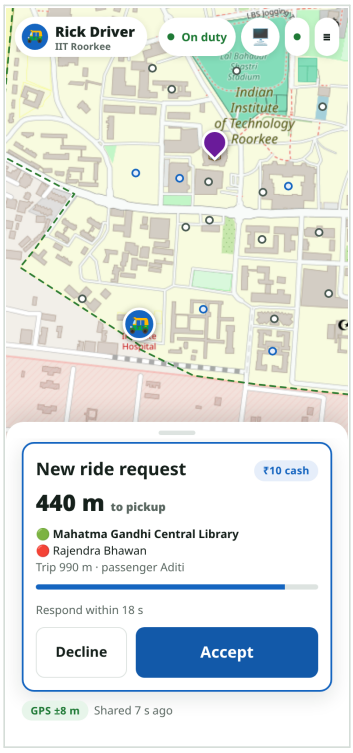
E-rickshaw booking, dispatch and live tracking for IIT Roorkee

Project report · Features · Path to production

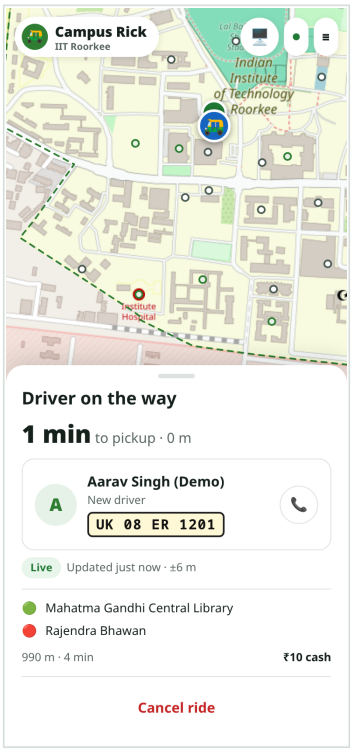
October 2026



Passenger: route preview and fare



Driver: incoming ride offer



Passenger: live driver tracking

Contents

1. Project basics	What Campus Rick is, who uses it, how it is built and how a ride flows
2. Core features	The features that make the service work end to end
3. Other features	Supporting features for passengers, drivers, administrators, security and quality
4. Path to production	Hosting, servers, OTP/SMS provider, costs, required work and launch checklist

1. Project basics

Campus Rick is a web-based booking and dispatch system for the e-rickshaws that run inside the IIT Roorkee campus. A passenger opens the app on their phone, signs in with a one-time code sent to their mobile number, picks a pickup and drop-off point on a campus map, and requests a ride for a flat fare. The system offers the ride to the nearest available driver, who accepts it in the driver app. The passenger then follows the rickshaw live on the map until the trip is completed and paid in cash. The transport office supervises everything from an admin console.

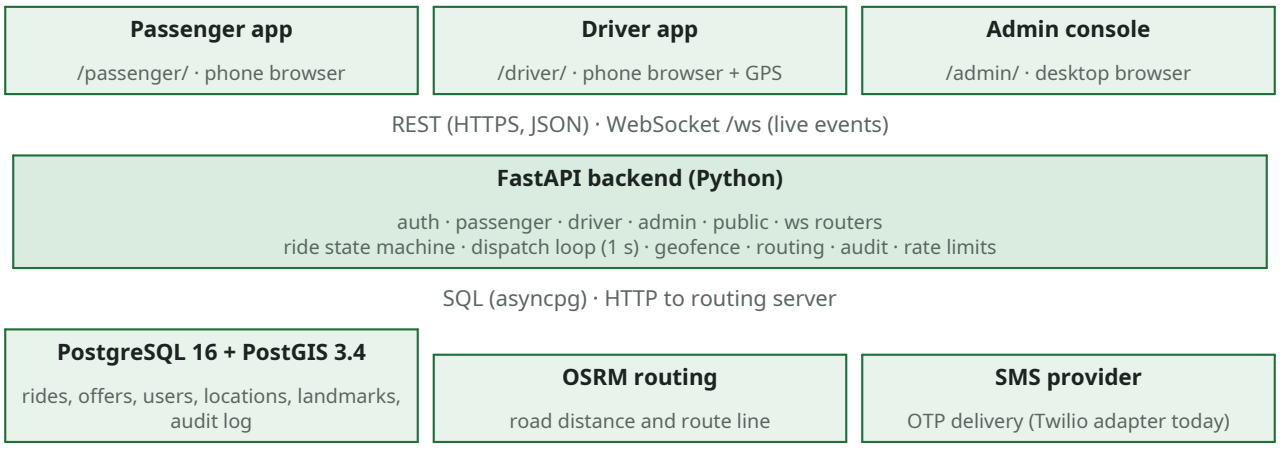
Who uses it

User	App	What they do
Passenger (students, staff, visitors)	/passenger/ on a phone	Sign in, book a ride, track the driver, rate the trip, see history
Driver	/driver/ on a phone	Go on duty, share GPS, accept offers, run the trip, collect cash, see earnings
Transport office (admin)	/admin/ on a computer	Watch live operations, verify drivers, manage campus places, set fares, read reports and the audit log

Technology

Layer	Choice
Frontend	Three browser apps written in plain JavaScript (ES modules), Leaflet maps with OpenStreetMap tiles. Installable on phones as web apps (passenger and driver apps ship a web manifest). No app store needed.
Backend	Python FastAPI with uvicorn, SQLAlchemy (async), Alembic migrations, JWT sessions, WebSockets.
Database	PostgreSQL 16 with the PostGIS 3.4 spatial extension (Docker container in development). SQLite is supported as a fallback for quick local runs.
Maps & routing	IIT Roorkee boundary and 39 campus landmarks taken from OpenStreetMap; road routes and distances from an OSRM routing server.
OTP delivery	Pluggable SMS provider. A Twilio adapter is written; development mode shows the code on screen instead.

How the system fits together



All three apps and the API are served by one backend process on one address (port 8091 in development). The apps use REST for actions and a WebSocket for live updates; when the socket drops they fall back to polling every 5 seconds and resynchronise when it reconnects.

Life of a ride

Every ride follows one state machine, enforced in the database so two screens can never disagree:

State	Meaning	Next step
REQUESTED	Passenger asked for a ride	Dispatch picks the nearest eligible driver
OFFERED	One driver has a time-limited offer (20 s by default)	Accept: ACCEPTED. Decline or timeout: next driver
ACCEPTED	Driver is heading to the pickup	Driver taps “I have arrived”
ARRIVED	Driver is waiting at the pickup	“Passenger on board — start trip”, or no-show after the wait time
IN_PROGRESS	Trip under way	Driver confirms cash received, then COMPLETED
COMPLETED	Trip finished and paid	Passenger and driver can rate each other
CANCELLED / NO_DRIVER_AVAILABLE	Cancelled by passenger, admin or no-show, or no driver found in time	—

Running it locally

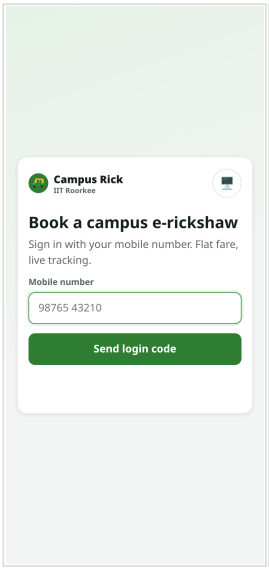
On Linux: `./start-erick.sh` (Windows: `start-erick.cmd`). The script creates the Python environment, starts PostGIS in Docker, applies migrations, loads campus data and demo accounts, and serves the apps at http://127.0.0.1:8091. Demo logins: passengers 8000000001–8000000015, drivers 9000000001–9000000010, admin 7000000001. In development the login code is shown on screen.

2. Core features

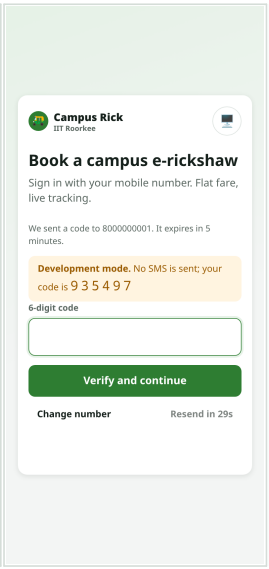
These are the features without which the service would not work. Screenshots are from a real run of the current code: one passenger and one driver phone completing a trip from the Central Library to Rajendra Bhawan.

2.1 Phone number sign-in with one-time code (OTP)

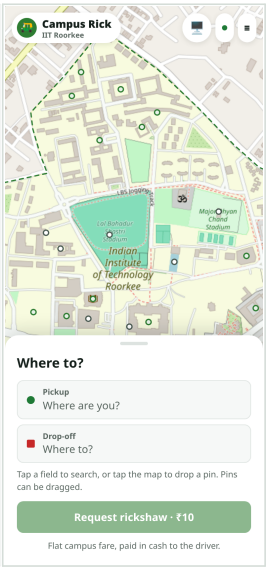
- Users sign in with their mobile number; a 6-digit code is sent by SMS. No passwords.
- Codes are stored hashed, expire after 5 minutes, allow 5 attempts, have a 30-second resend cooldown and are limited per number per hour and per IP address.
- Development mode shows the code on screen (yellow box) so the system can be tested without an SMS provider. The server refuses to start in production with development mode on.



Sign-in screen



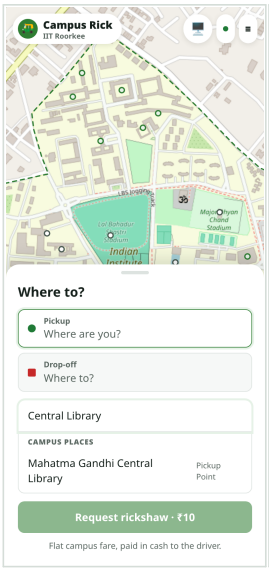
Code step (development mode shows the code)



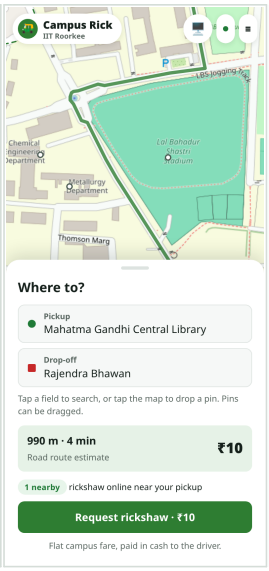
Home: campus map with landmarks

2.2 Booking a ride

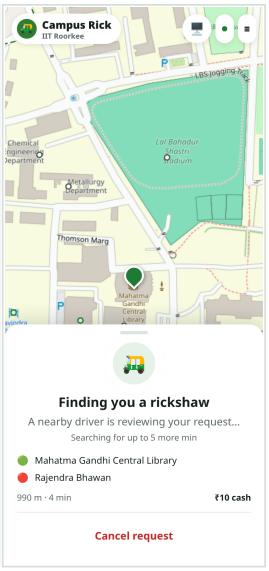
- Pickup and drop-off are chosen by searching the 39 campus places, tapping the map, or dragging a pin; “my location” uses the phone’s GPS.
- Points outside the campus boundary (plus a 150 m GPS tolerance) are rejected; trips shorter than 100 m are rejected.
- The route is drawn on the map with road distance and time from OSRM; if routing is unavailable a clearly labelled straight-line estimate is used.
- Flat fare (₹10 by default, set by the admin), shown before booking. The app also shows how many rickshaws are online nearby.
- Double taps cannot create two rides (idempotency key), and a passenger can only have one active ride at a time.



Searching campus places



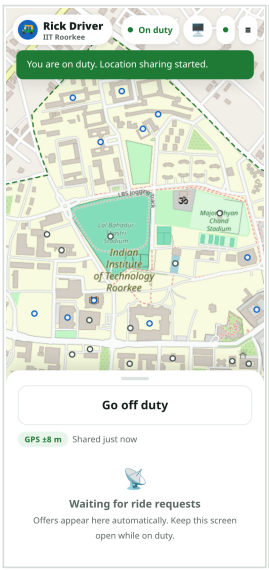
Route, time and fare preview



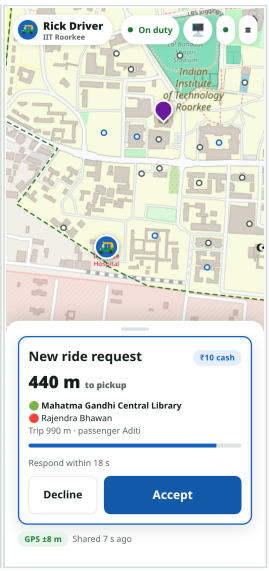
Finding a rickshaw

2.3 Automatic dispatch to the nearest driver

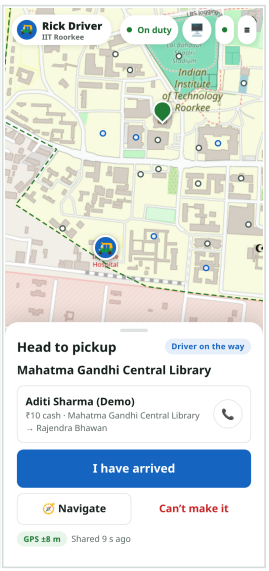
- Eligible drivers are on duty, approved, have a registered vehicle, a fresh GPS position inside campus, and no other ride or offer.
- The nearest one (PostGIS radius search, 2.5 km default) gets a time-limited offer with a countdown. Decline or timeout passes the ride to the next nearest driver; a driver is never offered the same ride twice.
- Accepting is atomic: if two drivers try at once, exactly one wins. A dispatch loop runs every second to expire offers and retry when new drivers come online; after 5 minutes or 8 offers the ride ends as “no driver available”.



Driver on duty, GPS shared



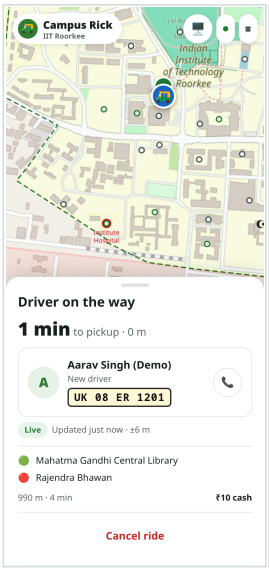
Offer with 20 s countdown



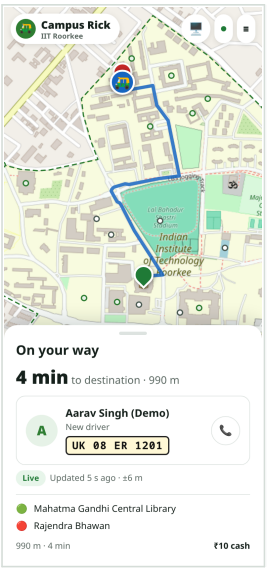
Accepted: head to pickup

2.4 Live tracking and real-time updates

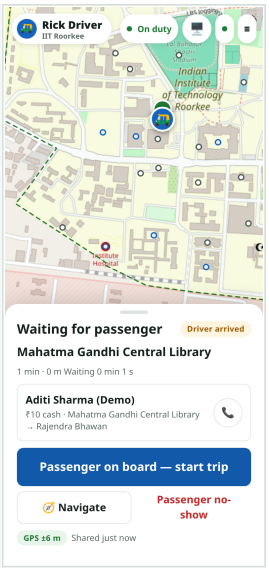
- The driver app sends its GPS position (at most every 3 s) while on duty. Stale, inaccurate, future-dated or out-of-order positions are rejected.
- The passenger sees the driver's name, vehicle number, a moving rickshaw marker, the ETA and a freshness label ("Live · Updated just now · ±6 m").
- All screens update instantly over an authenticated WebSocket; events are only sent after the database commit, so what you see always matches what is stored.



Driver on the way, live position



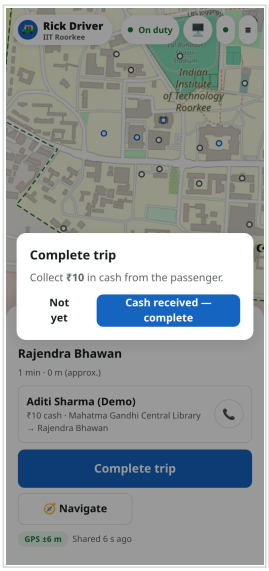
On the trip, route to destination



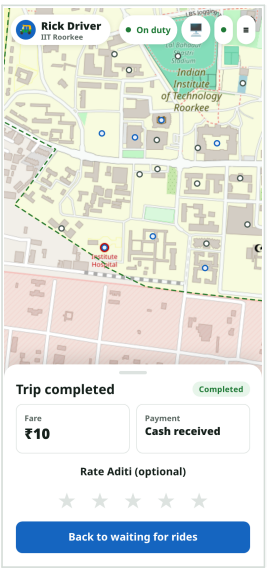
Driver waiting at pickup

2.5 Completing the trip and cash payment

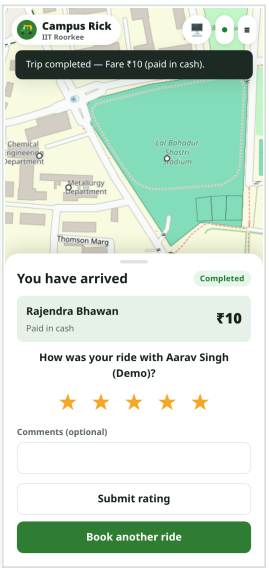
- The driver moves the trip forward with one large button: arrived, start trip, complete.
- On completion the driver confirms the cash amount received; the payment is recorded against the ride (amounts are stored in whole paise, so there are no rounding errors).
- Both sides see the completed trip, and the passenger can rate the driver with stars and a comment.



Driver confirms cash received



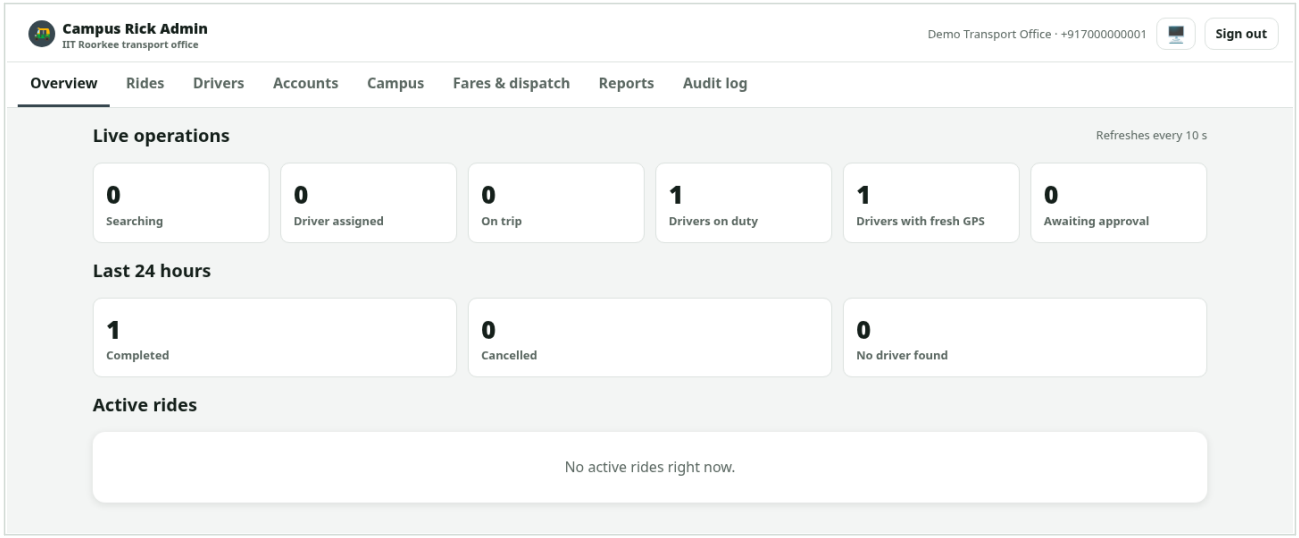
Driver: trip completed



Passenger: arrived and rating

2.6 Admin live operations

The transport office sees live counts (searching, assigned, on trip, drivers on duty, drivers with fresh GPS, awaiting approval), the last 24 hours, and all active rides. It refreshes every 10 seconds.

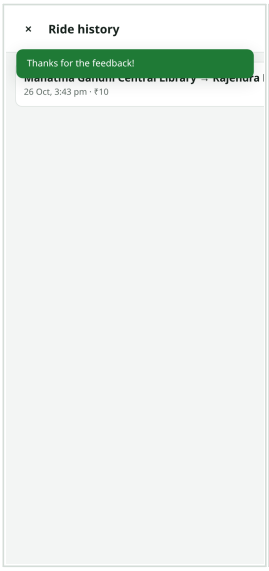


Admin console: live operations overview

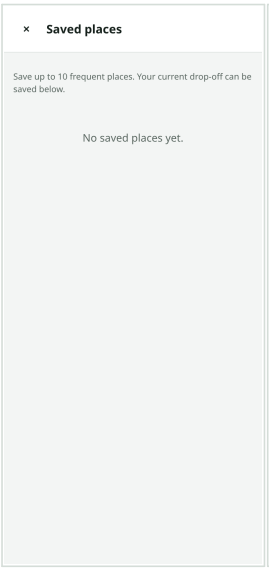
3. Other features

3.1 Passenger app

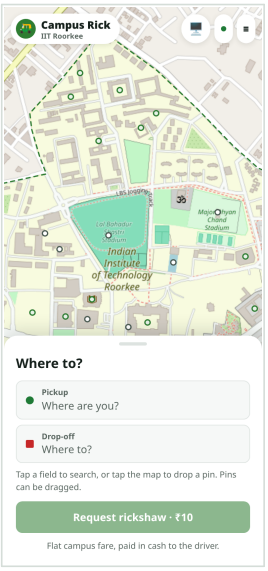
- **Ride history** with every past trip, fare and full status timeline.
- **Saved places** (up to 10) for quick booking, and a profile page.
- **Cancellation with a reason** while searching or before pickup; the offer disappears from the driver's screen immediately.
- **Ratings** (1–5 stars plus comment) after a completed trip.
- **Privacy:** while booking, nearby rickshaws are shown rounded to about 100 m; the exact driver position is shown only for your own trip. Driver contact details disappear once the trip ends.
- **Light and dark theme**, mobile-first layout, installable as a home-screen web app.



Ride history



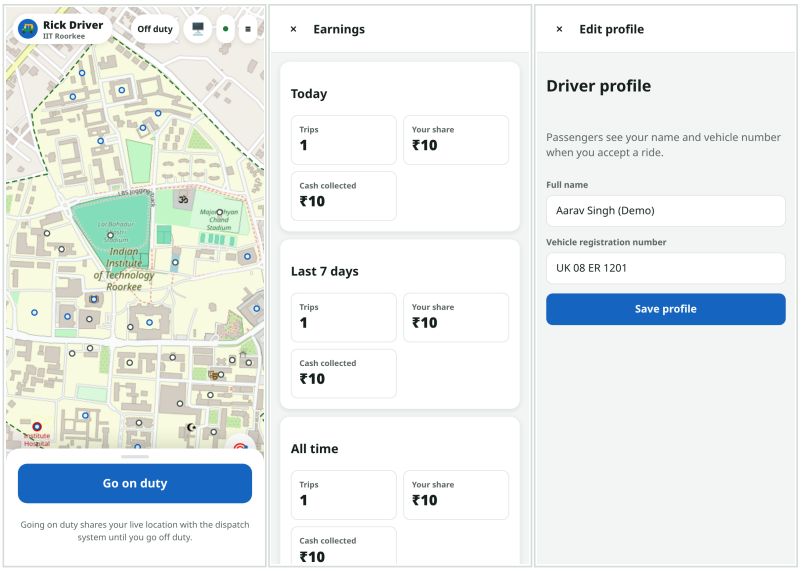
Saved places



Home screen

3.2 Driver app

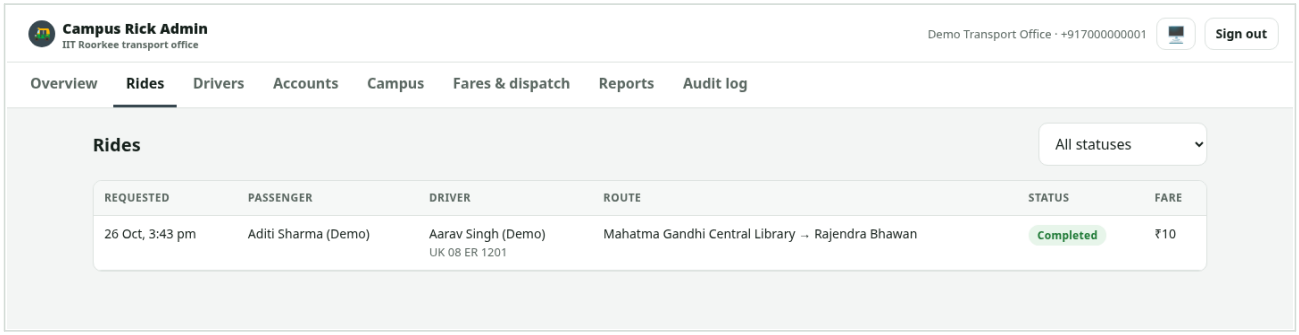
- **Profile and vehicle** (name, registration number) and verification status from the transport office.
- **Duty toggle** with explicit GPS states (waiting, shared, denied, inaccurate). Going off duty deletes the stored position. The app keeps the screen awake while on duty.
- **Navigation link** that opens the phone's maps app to the pickup or drop-off.
- **Release a ride** ("Can't make it") sends it back to dispatch for another driver; **passenger no-show** can be reported after the configured wait (3 minutes).
- **Earnings** for today, the last 7 days and all time, computed from completed rides and cash collected.
- **History** of past trips. A clearly labelled test simulator exists for development only.
- Before accepting, the driver sees only the passenger's first name and route; full name and phone appear after accepting.



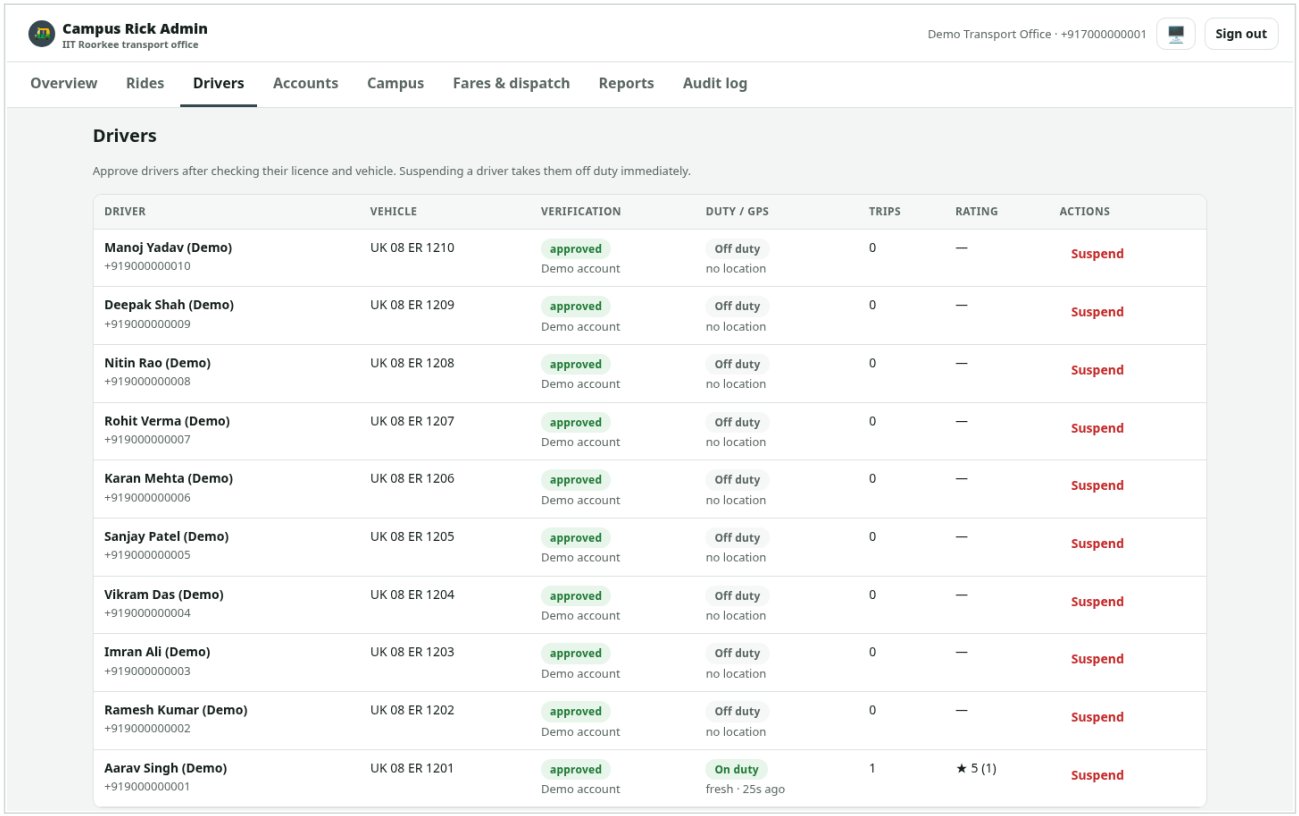
Off duty | Earnings summary | Profile and vehicle

3.3 Admin console

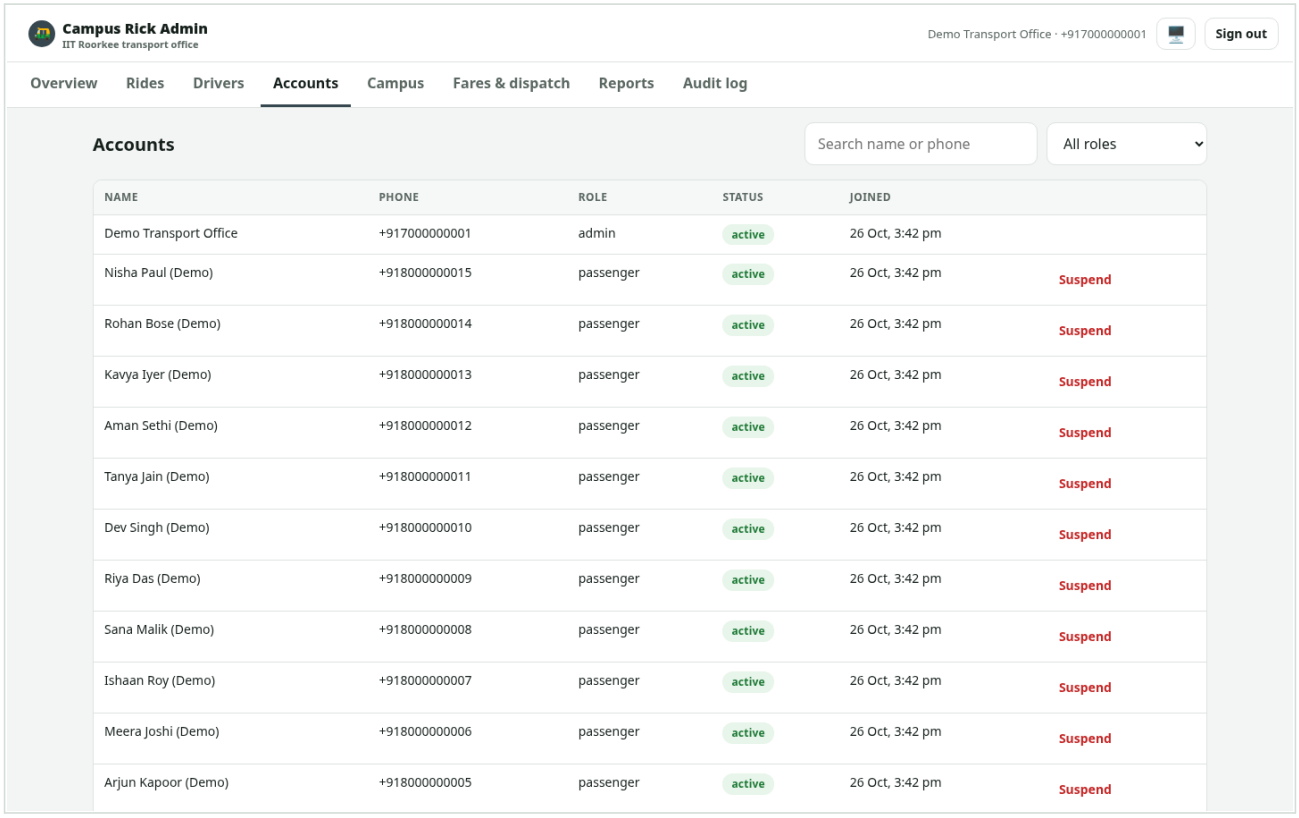
Tab	What it does
Rides	All rides with filters; ride detail, status timeline, offers sent, and admin cancellation.
Drivers	Approve or reject drivers, suspend them, see duty/GPS status, trips and rating.
Accounts	Search all users by name or phone; suspend accounts (sessions are revoked at once).
Campus	Campus boundary and landmarks on a map; add places, disable them, mark official pickup points.
Fares & dispatch	Flat fare, driver share, dispatch radius, offer timeout, max offers, search window, location freshness, arrival radius, no-show wait, require driver approval. Changes apply immediately and are audited.
Reports	Average wait to driver acceptance, trip duration, daily demand, busiest pickups, cancellations by role, driver offer outcomes.
Audit log	Every admin action with who, when, what and the target.



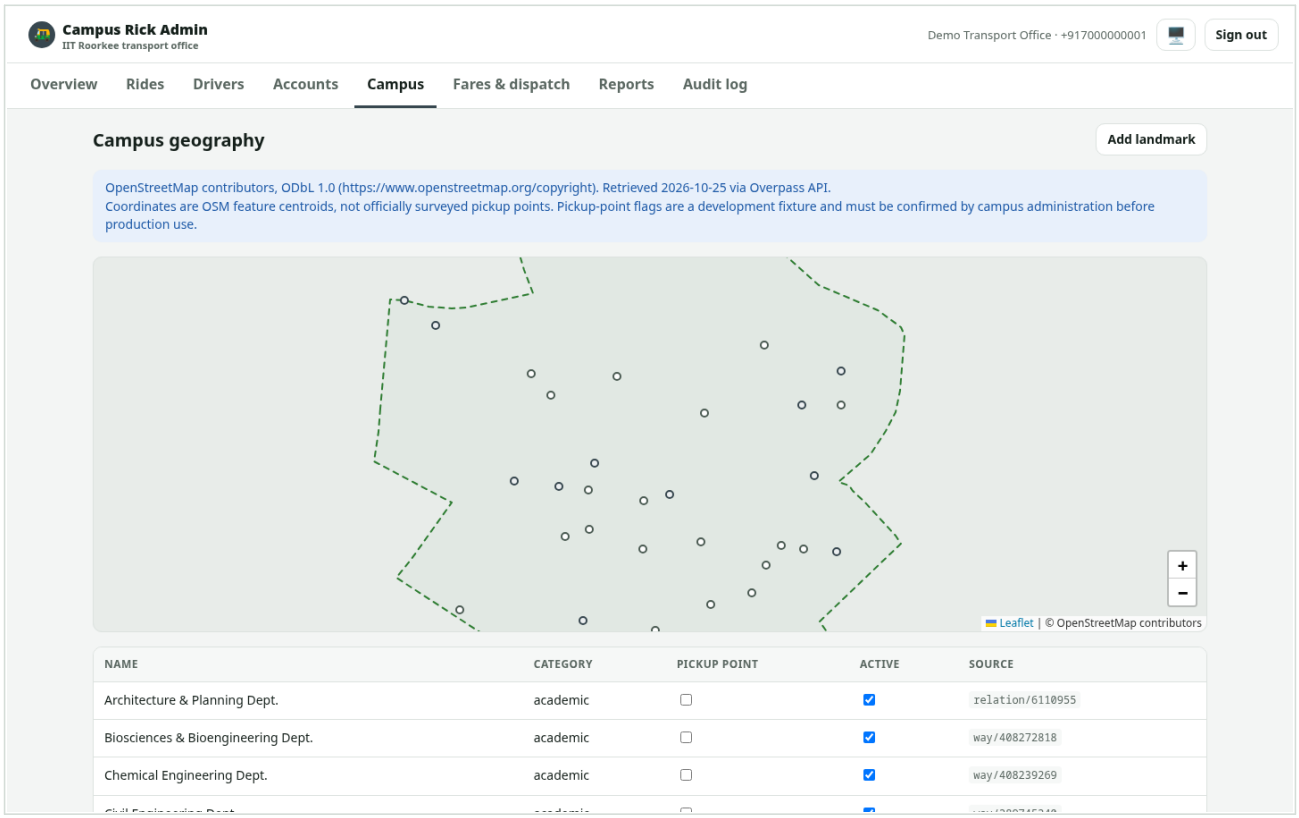
Admin console: Rides



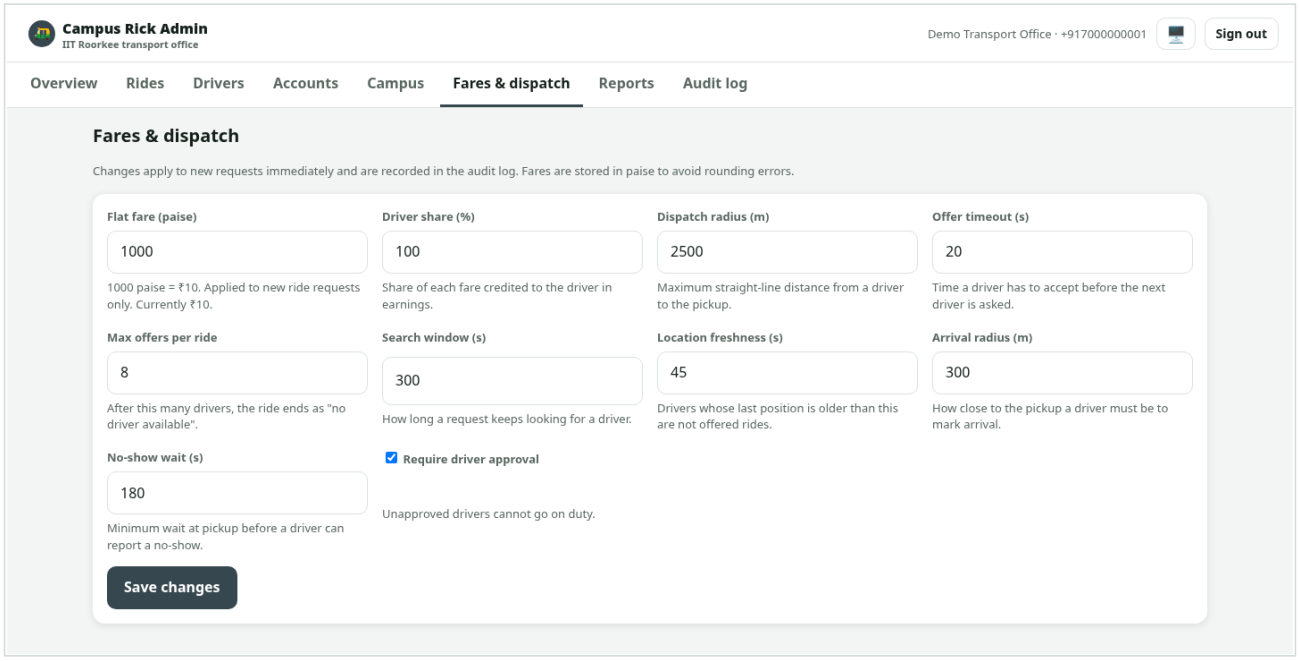
Admin console: Driver verification and suspension



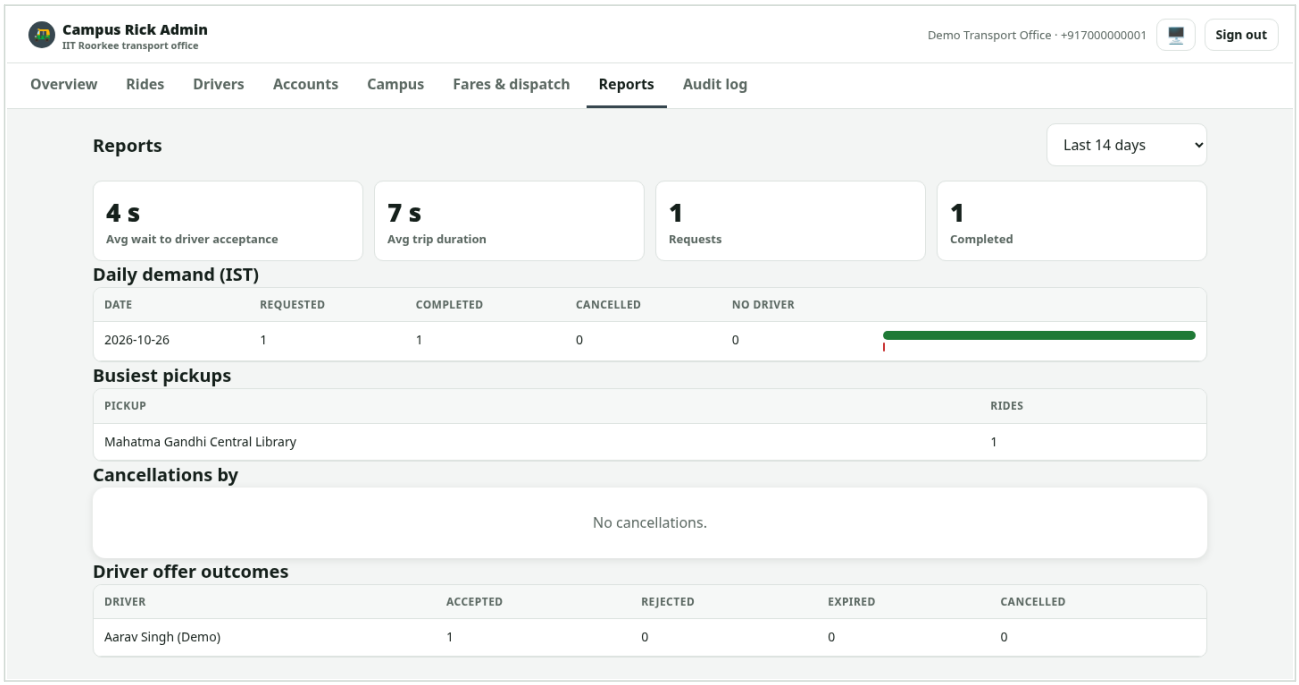
Admin console: Accounts



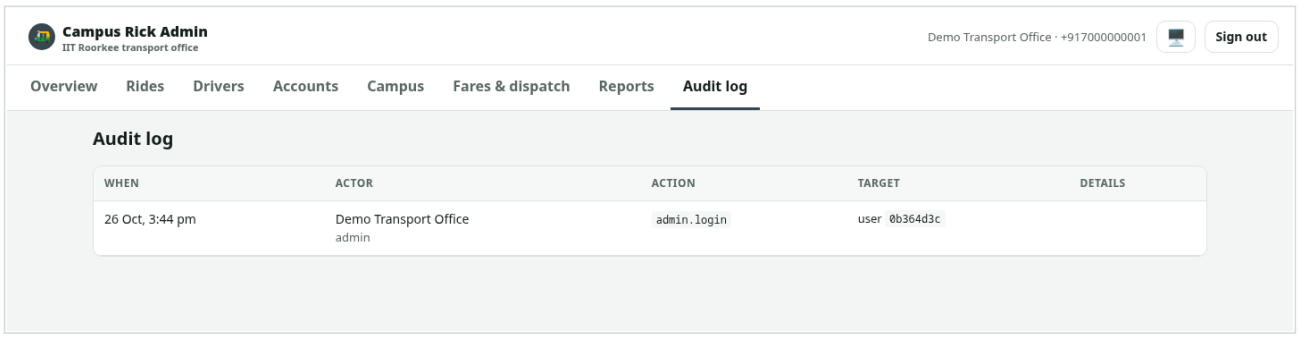
Admin console: Campus geography and landmarks



Admin console: Fares and dispatch settings



Admin console: Reports



Admin console: Audit log

3.4 Reliability, data integrity and security

- **Rules enforced by the database**, not just code: one active ride per passenger, one active ride per driver, one pending offer per ride and per driver, never re-offer a declined ride, one payment and one rating per trip.
- **Every state change** is a compare-and-set update recorded in a ride events table, so the history is complete and races are impossible.
- **Survives restarts:** all state is in PostgreSQL (tested); the old prototype lost everything on restart.
- **Security:** role checks on every endpoint, rate limits, security headers and a strict Content Security Policy, session revocation, short-lived tokens (24 h), suspended accounts disconnected within 30 s, and a production configuration guard that refuses unsafe settings.
- **Location privacy:** only the latest driver position is stored (no tracking history), and it is deleted when the driver goes off duty.
- **Tests:** 50 backend tests on PostgreSQL (49 + 1 skipped on SQLite) and a two-browser end-to-end test covering the full ride, cancellation, competing acceptance and the admin console.

4. Path to production

The application is feature-complete for a first launch, but it has only run on development machines. This section lists what has to be bought or arranged, the recommended hosting setup, the code work still required, and the launch steps.

> **Prices in this section are indicative** (October 2026, from vendor pages and comparison sites) and exclude GST. Confirm current rates with each vendor before committing.

4.1 What we need to get

Item	Why it is needed	Recommended option	Approx. cost
Server (VPS)	Runs the backend, database and routing server	1 Linux VPS in India: 2 vCPU, 4 GB RAM, 80 GB SSD (e.g. DigitalOcean Bangalore, AWS Lightsail Mumbai, or an IIT Roorkee institute server)	~US$20–25 / month; free if hosted by the institute
Domain name	Public address for the apps	Institute subdomain (e.g. rick.iitr.ac.in) or a .in domain	Free (subdomain) or ~₹800–1,500 / year
HTTPS certificate	Phones only allow GPS on HTTPS; also protects logins	Let's Encrypt via Caddy or nginx (auto-renewing)	Free
SMS / OTP provider	Sends login codes to phones	Indian DLT-compliant provider such as MSG91 (cheapest), or Twilio (already integrated)	MSG91 ~₹0.15–0.25 per OTP; Twilio ~US$0.083 (~₹7) per SMS
DLT registration	Required by TRAI for any business SMS to Indian numbers	Register the institute/entity, a 6-character sender ID and the OTP message template on an operator DLT portal (Jio, Airtel, Vi or BSNL)	One-time fee, typically a few thousand ₹ (varies by portal)
Backups storage	Off-server copies of the database	S3-compatible object storage (e.g. DigitalOcean Spaces, AWS S3)	~US$5 / month
Uptime monitoring	Alert when the service is down	UptimeRobot or Better Stack on /api/health	Free tier

Expected running cost for a campus-scale launch: about ₹2,000–3,000 per month for server and backups, plus SMS. Because sessions last 24 hours, each active user needs roughly one OTP per day: 300 daily users ~9,000 OTPs a month ~₹1,400–2,300 with an Indian provider (~₹63,000 with Twilio). Increasing the session lifetime (`JWT_EXPIRY_HOURS`) would lower SMS costs directly.

4.2 OTP / SMS: what to do

The code already supports real SMS: setting `SMS_PROVIDER=twilio` with the three `TWILIO_*` credentials makes the backend send codes through Twilio's Messages API. It has not yet been tested with real credentials. Recommended route:

- **Choose the provider.** For Indian numbers, an Indian provider (MSG91, Fast2SMS, Message Central or similar) is roughly 30–40× cheaper than Twilio and handles DLT natively. This needs a small new adapter in `backend/services/sms.py` (one function, about 20 lines, following the existing Twilio one).
- **Complete DLT registration** (entity, then sender ID header, then content template). Approval usually takes several days. The template must match the exact message the app sends: "{code} is your IIT Roorkee Campus Rick login code. It expires in 5 minutes."

- **Configure** `SMS_PROVIDER`, the provider API key and sender ID in the server's `backend/.env`, and set `OTP_DEV_MODE=false`.
- **Test** with a few real phone numbers on each major network (Jio, Airtel, Vi, BSNL) before launch.

4.3 Recommended hosting setup

The backend is designed to run as a **single instance** (live updates, rate limits and the dispatch loop live in one process), which is ample for a campus. Everything fits on one server:

Component	How it runs
Reverse proxy	Caddy (or nginx) on ports 80/443 with automatic HTTPS; forwards HTTP and WebSocket traffic to the backend on 127.0.0.1:8091.
Backend	FastAPI under uvicorn as a systemd service (auto-restart on failure and on boot), APP_ENV=production.
Database	PostgreSQL 16 + PostGIS 3.4 (the existing Docker image, or a managed PostgreSQL with PostGIS), not exposed to the internet.
Routing	Self-hosted OSRM with a small Uttarakhand / Roorkee map extract (the public OSRM demo server is rate-limited and not for production).
Backups	Nightly pg_dump to object storage, 14–30 days retained; test a restore before launch.
Monitoring	Uptime check on /api/health, server disk/CPU alerts, log rotation.

4.4 Work still required before launch

Task	Status / detail	Priority
Real SMS delivery	Twilio adapter written but untested; Indian provider adapter to be added; DLT registration	Must
Production configuration	Random JWT_SECRET (32+ chars), OTP_DEV_MODE=false, SIMULATION_ENABLED=false, real SMS_PROVIDER, PostgreSQL DATABASE_URL, CORS_ORIGINS if needed. The server refuses to start otherwise.	Must
Self-hosted routing	Set up OSRM and point ROUTING_BASE_URL at it	Must
Confirm pickup points	Landmarks are OpenStreetMap centroids; transport office should confirm and mark real pickup points in the admin console	Must
Deployment scripts	systemd unit, reverse proxy config, backup cron job (current scripts are for local development)	Must
Onboard drivers	Create driver accounts, register vehicles, approve them in the admin console	Must
OTP cleanup job	Purge OTP records older than 24 h (documented, not automated yet)	Should
Background GPS	Phone browsers pause GPS when the driver app is in the background or the screen is off; drivers must keep the app open. A native wrapper app would fix this.	Later
Push notifications	No notifications when the app is closed	Later
Online payments	Cash only today; a UPI/payment gateway could be added	Later
Scaling beyond one server	Needs Redis pub/sub and a single-leader dispatch loop	Later

4.5 Deployment steps

1. Provision the server (Ubuntu LTS), create a non-root user, enable the firewall (allow only SSH, 80, 443).
2. Point the domain's DNS A record at the server's IP address.
3. Install Docker (for PostGIS and OSRM), Python 3.13, and Caddy.
4. Copy the code to the server, create the virtual environment and install `backend/requirements.txt`.
5. Create `backend/.env` with production values (section 4.4) and the SMS provider credentials.
6. Start PostGIS, then run `python -m backend.manage migrate` and `python -m backend.manage load-campus`.
7. Create the first administrator: `python -m backend.manage create-admin --phone 98xxxxxxxx --name "Transport Office"`.
8. Start the backend as a systemd service and configure Caddy to proxy the domain to 127.0.0.1:8091 (HTTPS is automatic).
9. Set up nightly backups and the uptime monitor; check `/api/health`.
10. Pilot: a few drivers and a small group of passengers for one to two weeks, then open to the whole campus.

4.6 Launch checklist

	Check
☐	Domain resolves and HTTPS works on passenger, driver and admin URLs
☐	Real OTP received on Jio, Airtel, Vi and BSNL numbers; development mode is off (no code on screen)
☐	/api/health reports database ok, engine postgresql, dispatch loop running
☐	Route lines come from the self-hosted OSRM server
☐	Pickup points confirmed by the transport office; fare and dispatch settings reviewed
☐	All drivers onboarded, vehicles registered and approved; drivers trained to keep the app open on duty
☐	A full test ride completed on real phones on campus (book, accept, track, complete, rate)
☐	Backup taken and a restore tested; uptime alerts reach the team
☐	Demo accounts not present (seed-demo refuses to run in production)